

Object-oriented Programming 2 KU

Data Loading, XML Parsers (DOM, SAX)

Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)

Introduction to XML Parsers

What is an XML File?



- An XML file contains XML code and ends with the file extension “.xml”.
- It contains tags that define not only how the document should be structured but also, how it should be stored and transported over the internet.
- Used also in our project for Maven dependencies: pom.xml

```
<studentsList>
  <student id="1">
    <firstName>Greg</firstName>
    <lastName>Dean</lastName>
    <certificate>True</certificate>
    <scores>
      <module1>70</module1>
      <module12>80</module12>
      <module3>90</module3>
    </scores>
  </student>
  <student ind="2">
    <firstName>Wirt</firstName>
    <lastName>Wood</lastName>
    <certificate>True</certificate>
    <scores>
      <module1>80</module1>
      <module12>80.2</module12>
      <module3>80</module3>
    </scores>
  </student>
</studentsList>
```

DOM (Document Object Model) Parsers



- Loads the entire XML document into memory, creating a tree structure.
- Allows random access to any part of the XML.
- Suitable for smaller documents due to memory consumption.
- Efficient?:
 - No, if the document is large, as it requires loading the entire file into memory.
 - Optimization:
 - **Access the length of child nodes only once and store it.**
 - Frequent lookups are costly.
 - No multi-threading needed in general if done efficiently with HashMaps 😊
 - Fast enough for this assignment (if done correctly)
 - You may want to access the length of child nodes only once and save it in a variable as we are not restructuring the XML - your data loading might slow down significantly otherwise!

SAX (Simple API for XML) Parsers



- Reads XML sequentially, event-driven (pull-based), without loading the entire document.
- Memory-efficient, good for large files.
- No random access, must handle data as it's encountered.
- Efficient?:
 - Yes, especially for large files, since it doesn't load the entire structure into memory.
- Some tutorials:
 - [Oracle Docs](#)
 - [Baeldung](#)

Handling OSM Parsing

Handling OSM (Multipolygon) Relations



- You should read [this specification](#) carefully
- OSM ways and nodes do not correspond to roads and nodes - amenities can be ways too (and vice-versa) (as an amenity could be e.g. a closed way to indicate the building outline)
- All relations should be at least tried to be built (and succeed, there are no invalid ones!)
- OSM multipolygon relations define areas with complex shapes, including outer and inner boundaries.
- These are represented using relation elements that reference way elements.
- Must ensure outer boundaries form closed loops.
- Need to distinguish between holes (inner boundaries) and outer boundaries.

Approach with Parsers



- DOM Parser:
 - Easy to traverse the XML tree and build relations between nodes, ways, and multipolygons.
 - Be mindful of large OSM files—store relations in a hashmap for quick lookup to reduce traversal costs.
- SAX Parser:
 - Requires careful state management while parsing. Handle events related to relations and build them as the stream progresses.

Approach with Parsers



- General Parsing Tips:
 - Avoid redundant calculations. For DOM, store lengths and IDs in variables to prevent costly repeated traversals.
 - Use hashmaps for quick lookups when dealing with IDs like nodes or ways ($O(\log N)$ complexity).
- On-Demand Data Transfer:
 - Load data in chunks and only as requested by the middleware. This ensures efficient memory usage.
 - Make use of pagination to keep the data transfer manageable.
- Sorting and Consistency:
 - Always sort the OSM entities (e.g., nodes, ways) by ID in ascending order to maintain consistent pagination.

Approach with Parsers



- Memory and Performance Management:
 - Keep memory usage below ~3GB RAM. Preload only the necessary data in ~10 seconds.
 - SAX is recommended for large OSM files due to its memory efficiency.
 - However: DOM is probably easier to setup – You can choose whichever you prefer ☺
- MapLogger Usage:
 - Log all activities, even minor ones, to ensure traceability.
 - Utilize the provided logging methods (e.g., from MapLogger.java).
- Proper Exception Handling:
 - Use @ControllerAdvice in your Java backend to handle exceptions elegantly and consistently.
 - Implement the recommended environmental variables for backend connections and XML loading. (Similar to Assignment 0)



Example Code in TC ☺
Download [here](#)

Object-oriented Programming 2 KU

Data Loading, XML Parsers (DOM, SAX)

Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)